SmartHouse User Model and Requirements Analysis Design Model and Prototype

Luvo Goli
4 424 259
4 424 259@myuwc.ac.za

Lethabo Lethabo
4 141 210
4 141 210@myuwc.ac.za

Elam Mandulo
4 425 757
4 425 757@myuwc.ac.za

Lukhanyo Msothwane
4 426 268
4 426 268@myuwc.ac.za

Samkele Mthuli
4 418 450
4 418 450@myuwc.ac.za

Chuma Senzangabom
4 425 336
4 425 336@myuwc.ac.za

FivePlusOne Devs · CSC 312 Software Engineering · University of the Western Cape
Step 5 · 11 October 2026

ABSTRACT

SmartHouse helps prospective homeowners describe a home that fits their plot, household and budget, then explore a personalised concept. This report combines the user model, mental model and requirements analysis with the design and development of an executable frontend prototype. Project feedback and a review of two public product pages inform provisional user profiles and traceable requirements. Architectural, data, class, interface and component-level models explain the guided consultation, versioned briefs, local persistence, room generation, matching plan and 3D views, demo estimates and exports. A worked example connects room layout, plot fit and cost. The report records software engineering practices, verification coverage and evaluation needs. Independent participant research, authentication, professional approval and production costing remain future work.

1 INTRODUCTION

Early home planning involves more than choosing a drawing. A user must relate the size of the land, the spaces a household needs, the intended appearance and the money available. SmartHouse proposes a guided consultation that produces a confirmed brief and a house concept with matching views and estimated costs [1]. The problem is framed as helping users make understandable early design choices, rather than guaranteeing a construction price or replacing professional approval.

The user model and requirements analysis establish who uses the system, what users expect the interface to represent, and which data, actions and responses follow from their goals [2]. This report extends that analysis with the Step 5 design model and prototype development required by the course [9], using the supplied report format [6]. Sections 2-6 retain the analysis foundation; Sections 7-10 explain the design, executable prototype, engineering practices and evaluation. The design baseline is the frontend prototype documented on 11 October 2026 [8].

Rectangular plots and single-storey homes form the initial supported scope. Complex terrain, detailed engineering, procurement and approved construction drawings are excluded. The current application runs in the browser without a login, API server or database. Future platform requirements are distinguished from implemented prototype behaviour throughout the report.

2 EVIDENCE AND ELICITATION

2.1 Evidence sources

Three source types inform the analysis. First, the revised proposal defines the product purpose and stakeholders [1]. Second, the project discussion records feedback from a product owner or team representative about personalisation, consultation, visualisation and excessive scrolling [3]. Third, a desk review on 11 October 2026 examines the public product pages of Planner 5D and Floorplanner [4, 5]. The review records advertised functionality; it does not establish product usability, price accuracy or successful use by participants.

No independent interview transcripts, survey responses or timed participant observations are analysed here. Accordingly, the profiles and requirements are hypotheses grounded in this evidence, not demographic findings or measured population preferences. Table 1 records the feedback and its interpretation. Evidence identifiers allow requirements to be traced to their basis.

Table 1: Project feedback and interpretation

ID	Recorded need	Interpretation
E1	A different house tailored to each user.	Collect household and lifestyle needs, not only plot area and budget.
E2	Ask questions an architect would ask.	Use a guided consultation and reviewable brief.
E3	See the house, not just the floor plan.	Provide consistent exterior and interactive 3D views.

ID	Recorded need	Interpretation
E4	The project page requires too much scrolling.	Separate brief editing from design exploration; compact controls.
E5	Build a prototype without backend/authentication.	Support a local demo without sign-in or API dependency.

2.2 Comparative desk review

Planner 5D presents floor-plan creation, furnishing and visual rendering on its public page [4]. The relevant observation is that a plan and a visual appearance are complementary representations. Floorplanner advertises room, wall, door and window editing together with a real-time 3D view [5]. This supports investigating whether users understand the relationship between layout choices and the visible house. These observations are secondary evidence, not proof that SmartHouse users need every advertised feature. Table 2 records the design implications.

Table 2: Desk review collected 11 October 2026

Source	Observation	Implication
E6 / [4]	Public page links floor plans and visual rendering.	Keep plan and appearance connected.
E7 / [5]	Public page describes openings and real-time 3D.	Inspect doors, windows and rooms in consistent views.

2.3 Interpretation and validation needs

Taken together, E1-E7 suggest that the main value is a comprehensible personal design process: explain needs, confirm them, see the proposed house, understand tradeoffs and revise. This is an interpretation to test. Research must establish which questions users can answer reliably, whether plan dimensions are understood, what costs users assume are included, and whether the revised project navigation reduces effort.

3 USER MODEL

3.1 Provisional user profiles

A first-time homeowner may have little experience reading architectural plans. This user needs familiar room names, units, examples and visible assumptions. The main tasks are to enter site and budget information, describe essential accommodation, inspect an understandable house concept, and recognise when a requirement exceeds the supported scope. Architectural literacy is a design concern, not a measured characteristic of a recruited sample.

A household planner may be deciding for adults, children, visiting relatives or future household changes. This user needs to express bedrooms, bathrooms, privacy, storage and guest accommodation and to distinguish essentials from optional features. A home-working user may require an office and separation from noise. A user with access needs may require step-free movement or other accommodations; those needs must be recorded explicitly and reviewed rather than inferred from age or household type.

A demo manager maintains illustrative catalogue items and rates. In the prototype, this is an interface role available without secured permissions, not a protected administrator account. In the later platform, an administrator would manage published design/pricing data and access. Architects/designers and cost practitioners are secondary stakeholders who may review exported concepts and assumptions. No professional login or approval workflow is implemented. Table 3 relates the profiles to their tasks.

Table 3: Users and tasks

Profile	Tasks and needs
Homeowner	Start/resume a project; describe site and budget; confirm a brief; inspect and export a concept.
Household planner	Describe occupants, privacy and future needs; prioritise rooms; revise requirements.
Home-working / access needs	Record office/access requirements; identify support limits and review needs.
Demo manager	Maintain illustrative designs/rates locally; understand that changes affect demo data.

3.2 Use case model

Figure 1 models the users and their goals inside the SmartHouse boundary. Validation and generation are system responses within these tasks. Professional review is outside the current application boundary and is not shown as an implemented approval use case. Table 4 identifies the goals; the written cases in Section 6 expand key interactions from the user perspective.

Table 4: Use case catalogue

ID	Goal	Result
UC1	Create/resume project	Named workspace or restored draft.
UC2	Provide needs	Saved consultation answers.
UC3	Confirm brief	Reviewed requirement snapshot.
UC4	Generate concept	House option with conflict feedback.
UC5	Inspect views	Understand plan, exterior and 3D.
UC6	Revise needs	Updated brief and concept.
UC7	Export concept	SVG plan or JSON summary.
UC8	Maintain demo content	Updated local catalogue/rates.

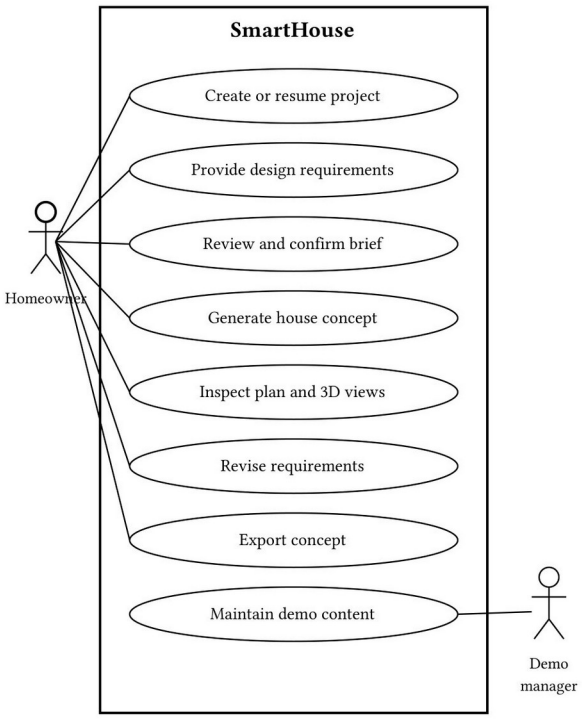


Figure 1: UML use case view of homeowner and demo manager tasks.

4 MENTAL MODEL

4.1 Architectural consultation metaphor

The intended mental model is an architectural consultation followed by reviewing a sketchbook and a model house. A project is the user's client file. The consultation is the conversation about land, family, daily life and money. The brief is the agreed description of the desired home. The concept is a proposal to inspect and revise. This metaphor explains why users answer questions before seeing the house, and why confirming a brief should not be mistaken for approving construction.

Figure 2 shows how the project file, consultation and confirmed brief lead to a concept. Floor plans and 3D views are different views of the same proposed house. The budget acts as an envelope for design choices; the interface should make the cost effect of revisions visible. A roof-removal control uses the familiar idea of a dollhouse to expose room relationships without requiring CAD terminology. Table 5 records the expected meanings of the interface terms.

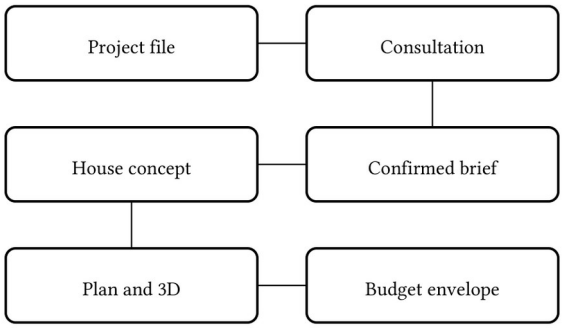


Figure 2: Familiar concepts that organise the design journey.

Table 5: Interface concepts and familiar meanings

Term	Mental image	Expected behaviour
Project	Client folder	Resume the same home discussion.
Brief	Agreed needs	Review/edit before generation.
House concept	Architectural sketch	Explore and request changes.
3D / cutaway	Model house	Rotate, zoom and inspect rooms.
Estimate	Planning budget	See basis, exclusions and tradeoffs.
Version	Sketchbook entry	Retain a confirmed brief after revisions.

4.2 Interaction principles

The interface should place the user in control: allow back navigation, show editable summaries and retain saved progress. Memory load should be reduced through labelled units, conditional questions and preserved answers. Consistency should link room names, counts and dimensions across views. Feedback should distinguish saving, confirmation, generation, conflict and unsupported scope. E4 specifically motivates a separate design workspace and brief editor, instead of presenting a long form above the house.

An attractive 3D scene can appear more authoritative than the underlying estimate or layout warrants. Therefore, the concept stage, pricing assumptions and matters requiring review must be understandable. In the prototype, browser-local saving is also made explicit so that users do not assume cross-device access.

5 REQUIREMENTS ANALYSIS

5.1 Data model at analysis level

The analysis identifies domain information independently of an implementation technology. Figure 3 relates a project to design briefs, site/budget information, house concepts, spaces/openings

and estimates. A household description and requirement set inform a brief. A concept expresses rooms and appearance and is presented as a plan or model. Section 7 refines these abstract relationships into storage records, generated objects and component responsibilities.

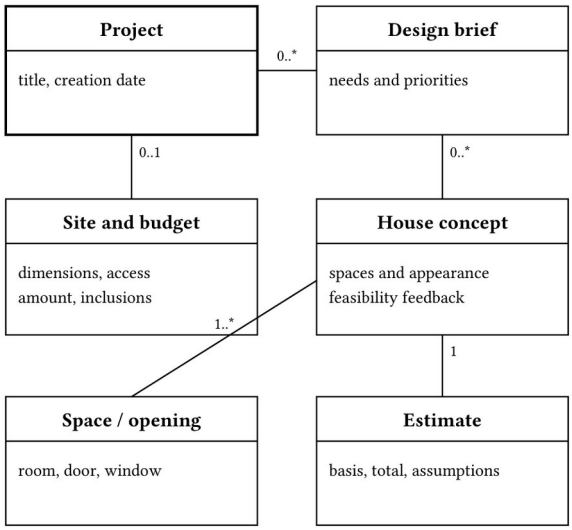


Figure 3: Abstract class relationships for requirements analysis.

The current prototype saves drafts and confirmed briefs locally and derives geometry for viewing. It does not persist production-grade mesh or price snapshots. Figure 3 represents required domain relationships rather than claiming that every entity has a database table. Table 6 records the data and constraints that follow from the user tasks.

Table 6: Data requirements and constraints

Data	Required information and rules
Site	Width, length, supported shape, road access, location and known terrain; unknown facts remain explicit.
Budget	Positive amount, currency, intended inclusions and finish preference; no implied guaranteed price.
Household / needs	Occupants, room counts, lifestyle, privacy, accessibility, essentials and optional preferences.
Brief / version	Reviewed answers, assumptions, support limits and version identity.
Concept / views	Footprint, room/opening geometry and appearance; shared identity across plan and 3D.
Estimate / export	Calculation basis, rate context, total and exclusions; export identifies the concept/brief.

5.2 Functional requirements

Table 7 uses report-specific identifiers, separate from the broader proposal requirements. Must and Should describe priority. Status distinguishes implemented prototype behaviour (I), partial support (P) and later platform work (L). Evidence is drawn from Tables 1 and 2 and the proposal [1]. Technical implementation alone does not validate the user need.

Table 7: Functional requirements and traceability

ID	User-facing requirement	Basis	Priority/ status
F01	Create, name and resume a project.	E2, [1]	Must / I
F02	Complete guided questions with relevant follow-ups.	E1, E2	Must / I
F03	Record household essentials and optional needs.	E1, [1]	Must / P
F04	Review, edit and confirm the design brief.	E2	Must / I
F05	Generate a personalised spatial concept.	E1	Must / P
F06	Explain plot or estimated-budget conflicts.	[1]	Must / P
F07	Keep plan, exterior and 3D geometry consistent.	E3, E6, E7	Must / I
F08	Rotate/zoom the house and inspect its interior.	E3, E7	Must / I
F09	Show the estimate basis and relevant exclusions.	[1]	Must / P
F10	Revise needs and retain confirmed brief versions.	E1, E2	Must / P
F11	Download a concept/plan for later discussion.	[1]	Must / P
F12	Maintain illustrative demo designs and rates.	[1]	Should / I
F13	Protect and synchronise user-owned projects.	[1]	Later / L

Partial status is significant. The generator uses one adaptable room pattern; extra features may be recorded without being generated. Plot/budget checks use demo assumptions. Costing uses illustrative area rates rather than verified construction quantities. Brief snapshots are retained but geometry is recomputed. Exports provide SVG/JSON rather than a complete professional package. F13 is deliberately outside the login-free prototype scope.

5.3 Nonfunctional requirements

The quality requirements in Table 8 are acceptance proposals, not measured study results. Evaluation must record test conditions, devices and participant tasks before asserting that a quality target has been achieved.

Table 8: Quality requirements and proposed evaluation

ID	Quality requirement	Acceptance evidence
N01	Understandable consultation and navigation.	Representative users complete the agreed core tasks and explain their next action.
N02	Keyboard/touch access and readable content.	Manual checks of labels, focus, contrast, camera buttons and narrow-screen use.
N03	Consistent visual representations.	Geometry/version checks; no room-count or opening mismatch between views.
N04	Reliable saving and recovery.	Resume tests, stale-revision rejection and clear storage-failure feedback.
N05	Responsive interaction and bounded work.	Documented generation/viewer benchmarks; targets agreed before evaluation.
N06	Transparent estimates and limits.	Users identify demo rates, exclusions and concept-stage status.
N07	Maintainable system responsibilities.	Separate consultation, geometry, visualisation, cost and persistence contracts.
N08	Appropriate privacy in each release.	Local-storage disclosure now; access/security tests before account-based release.

5.4 Behaviour and business rules

The main states are new project, editing draft, saved draft, confirmed brief, concept exploration and revised draft. Confirming a revised draft creates a new brief snapshot. Saving is not the same as confirming. An invalid input returns to a correctable field; a stale revision requires reconciling the latest saved draft. A generation conflict must not silently remove an essential room. A failed 3D view offers a two-dimensional fallback rather than blocking the whole task. Table 9 defines the event responses.

Table 9: Events and required responses

Event	Required response
Save incomplete draft	Validate supplied values, preserve progress and allow later completion.
Confirm brief	Check required/conditional answers and acknowledgements; explain unsupported scope.
Show concept	Present matching views, cost basis and plot/budget conflicts.
Edit confirmed needs	Retain the previous brief; show that the existing concept reflects the old confirmation.
Storage/graphics failure	Show actionable recovery or fallback; do not report a false success.

Site dimensions and budget must be finite and positive. Room counts must fall within supported limits. Unknown site conditions remain labelled. Prototype scope is rectangular, single-storey planning; stated sloped terrain blocks confirmation. The budget comparison is conditional on demo rates and exclusions. Building compliance and guaranteed construction affordability are not outcomes of this prototype.

6 WRITTEN USE CASES

6.1 UC3 Confirm the design brief

Goal: agree on the information used to produce the house. Preconditions: a project exists and the user has entered consultation answers. Main flow: (1) the user opens Review brief; (2) checks site, household, budget and style summaries; (3) edits an inaccurate section; (4) acknowledges concept assumptions; (5) confirms. The system checks required and conditional values, saves the brief snapshot and opens the design workspace. Alternate flows: missing answers lead to the relevant questions; unsupported scope is explained while the draft is preserved; a stale revision requires reload/reconciliation. Postcondition: a confirmed brief exists, or the editable draft remains without a false confirmation.

6.2 UC5 Inspect the proposed home

Goal: understand how the house looks and how its spaces connect. Preconditions: a confirmed brief and generated concept exist. Main flow: (1) the user opens Your house; (2) rotates/zooms the interactive model; (3) selects front, overview or above views; (4) removes the roof to inspect rooms; (5) opens the floor plan or exterior views; (6) reads cost and plot feedback. Alternate flow: unavailable 3D graphics provide a static exterior fallback. Postcondition: the user can inspect the same concept in available representations and see unresolved constraints; no claim of professional approval is made.

6.3 UC6 Revise household requirements

Goal: change the concept when needs or priorities change. Preconditions: a project and confirmed brief exist. Main flow: (1) the user opens Edit brief; (2) changes a room count or supported preference; (3) saves the draft; (4) reviews the revised summary; (5) reconfirms; (6) inspects the updated concept and estimated budget effect. Alternate flows: the user can pause after saving, or retain the previous confirmed concept if confirmation fails. Postcondition: a new brief snapshot is retained and the current views reflect it. Exact historical geometry/pricing replay is a later requirement, not a completed capability.

6.4 UC7 Export a concept

Goal: keep information for later discussion. Preconditions: a confirmed concept is displayed. Main flow: (1) the user chooses Download floor plan or Download concept data; (2) the system packages the corresponding SVG or JSON; (3) the user saves the file; (4) uses it in subsequent discussion. Alternate flow: a

missing render or failed download must not be reported as a successful export. Postcondition: an available local concept file identifies the brief and assumptions. A PDF concept package and professional drawings are later extensions.

7 DESIGN MODEL

7.1 Architectural design

The active design is one React and Vite browser application with local persistence and generated geometry [8]. Figure 4 separates the screen layer, local data operations, generation and representations. ProjectsPage, ConsultationPage, PrototypeConcept, Library and Manage demo provide the main screens. The compatibility service api.js exposes get/post/put methods as local function calls; these are not HTTP requests. The local adapter in store.js validates drafts, confirms brief versions and updates browser-local demo state.

PrototypeConcept derives a house from the confirmed brief when the concept view renders. The layout.js module creates rooms, a corridor, openings, the footprint, a demo estimate and conflict messages. All visual representations consume the same generated house object. SVG rendering supplies plans and dimensions; houseModel.js and the 3D/exterior components use the same geometry for visual exploration. Cost feedback uses the generated area and estimate fields. This shared representation supports F07 and N03 by keeping views tied to the same room and opening dimensions.

The browser stores the brief, not a versioned mesh file. Three.js creates meshes from the generated dimensions. A legacy Express/PostgreSQL implementation and target backend specifications remain in the repository, but neither is part of the active frontend run command or data path [8]. Separating these boundaries prevents future authentication and server requirements from being mistaken for current capabilities.

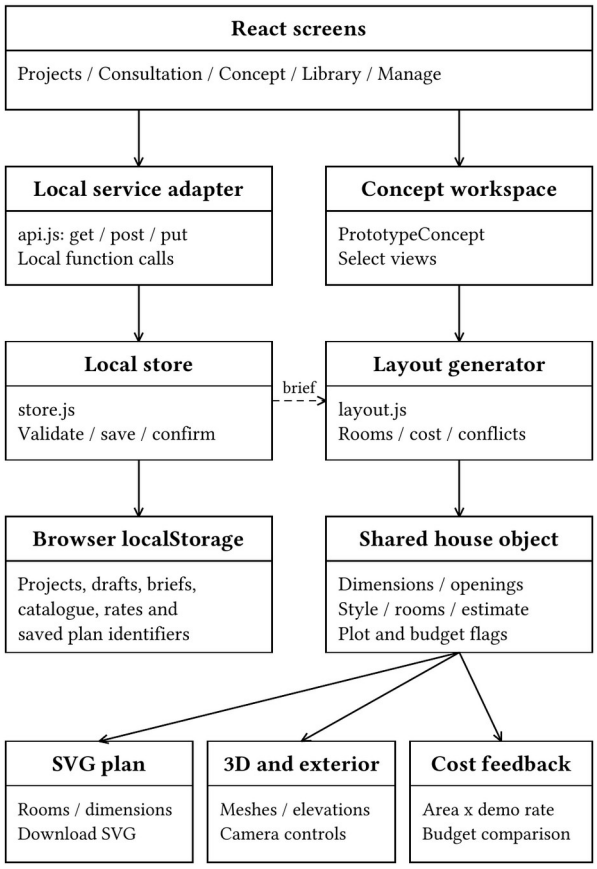


Figure 4: Active browser architecture and shared concept data.

7.2 Data design and persistence

Data design distinguishes persistent project state from derived geometry. Browser localStorage uses the key smarthouse.frontend-prototype.v1 to retain projects, drafts, confirmed briefs, catalogue plans, demo rates, saved plan identifiers and the last catalogue search. Figure 5 refines Figure 3 into the prototype's object relationships. The current storage model is a local document structure; a relational database and production ERD are future platform concerns.

A Project has an identifier, title, creation timestamp, one mutable Draft and zero or more ConfirmedBrief records. The Draft stores a revision number and a question-key/value answer map. Saving increments its revision. Each successful confirmation retains an answer snapshot together with its project identifier, version, draft revision and confirmation time. The confirmed brief also carries site area, currency and stage information. These fields distinguish a saved working answer set from a reviewed version.

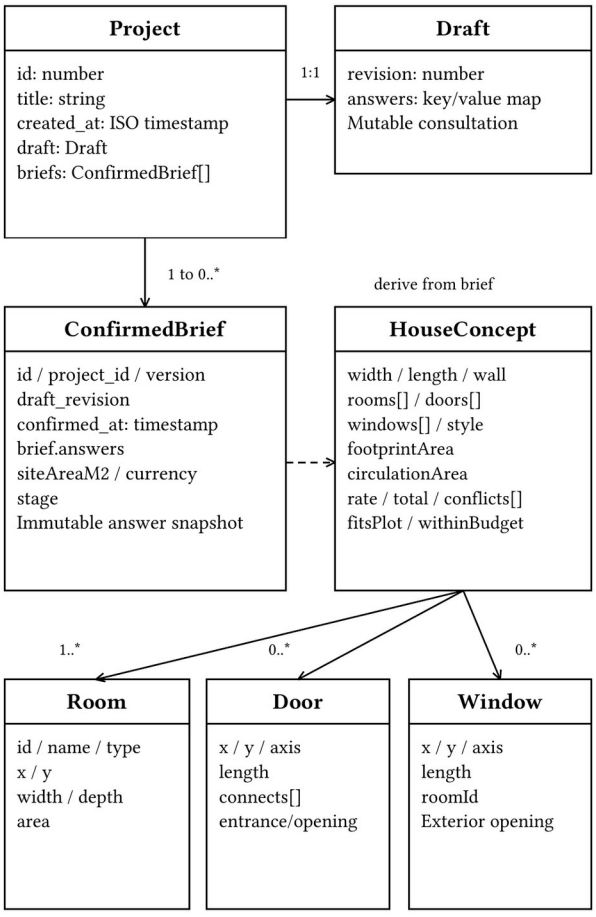


Figure 5: Detailed record and object model with cardinalities and lifecycles.

HouseConcept is derived in memory. Its dimensions, rooms, doors and windows support the plan and model; its area, rate, total and conflict flags support feedback. A Room records identity, name/type, coordinates, dimensions and area. A Door records position, axis, length and connected spaces or entrance/opening role. A Window records position, axis, length and its room identifier. Figure 5 describes JavaScript data objects and ownership relationships, without implying that each box is an implemented class declaration or database table.

Integrity rules follow the data lifecycle. Draft updates must reject a stale expected revision; confirmation must validate required and conditional answers and retain the prior confirmed brief. Storage errors must produce visible failure feedback. No account ownership or cross-device synchronisation is active. Geometry is recomputed using the current generator, so retained briefs do not guarantee exact historical layout replay. Custom concept rates are fixed demo values in layout.js; edits to library rates affect catalogue estimates. A later snapshot model must explicitly pin geometry, rate values and configuration versions.

7.3 Class and component responsibilities

The design assigns behaviour to modules around the data objects, rather than placing all operations on Project. The local store owns create/save/confirm/persist behaviour; the generator owns spatial construction and conflicts; rendering components own their representations. Table 10 states design-level operation contracts for these responsibilities. The names express pseudocode interfaces and are not claims about exact exported function names.

Table 10: Design operation contracts

Responsibility	Inputs and outputs
Save draft	projectId, answers, expectedRevision → updated Draft or validation/revision/storage error.
Confirm brief	projectId, current draft, acknowledgements → ConfirmedBrief or correctable error.
Generate concept	ConfirmedBrief → HouseConcept with geometry, estimate and conflicts.
Build 3D model	HouseConcept → scene objects with matching rooms/openings and style.
Render/export	HouseConcept and brief context → SVG plan or JSON concept data.
Dispose viewer	Mounted viewer resources → released geometry, materials, controls and renderer.

The design keeps persistence, generation and display responsibilities distinct, supporting N07. A view change does not need a new saved brief. A revised answer set does require review and confirmation before it becomes the current concept input. When a view unmounts, its resource lifecycle ends independently of the stored project.

7.4 Interaction and state design

Figure 6 expands UC3 into the interactions between user, interface, adapter, storage and concept views. Draft saving validates supplied values and persists the next revision. Confirmation checks required and conditional answers, the supported site scope and acknowledgements before saving a brief snapshot. The confirmed brief then drives local geometry and visible feedback.

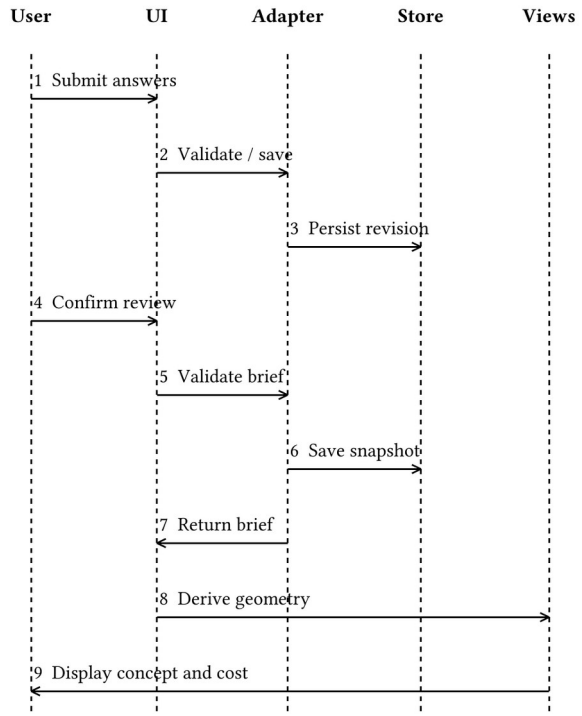


Figure 6: Local save, confirmation and concept display sequence.

Draft-to-saved-draft transitions preserve answers and increment the revision; stale expected revisions are rejected. Saved-draft-to-confirmed-brief transitions require complete answers, support for a rectangular plot and no stated sloped terrain. Confirmed-brief-to-workspace transitions generate a concept locally and retain plot/budget conflicts. Editing and reconfirming creates a new snapshot while preserving the previous confirmed brief. These transitions implement the event rules in Table 9 without silently removing requested rooms when a constraint fails.

7.5 Component level design

The layout generator uses a constrained spatial pattern [8]. A front living zone connects to a central hallway 1.2 m wide. Bedrooms, bathrooms and an optional dedicated office occupy paired bays. An unmatched bay becomes laundry/storage. When a separate kitchen is requested in the supported pattern, it has a direct opening to the living area. Fixed component sizes and wall/circulation allowances determine the outside footprint.

The following pseudocode summarises the documented save and confirmation rules. It expresses the design flow rather than reproducing source code.

Save and confirm a brief

```
SAVE_DRAFT(projectId, answers, expectedRevision)
  project ⟵ load local project
  if expectedRevision ≠ project.draft.revision
    return stale revision error
  validate supplied values; allow incomplete answers
  nextDraft ⟵ answers and revision + 1
  persist nextDraft; report any storage failure
  return saved draft only after persistence succeeds

CONFIRM_BRIEF(projectId, reviewedDraft, acknowledgement)
  check latest draft revision and required answers
  check conditional questions and acknowledgement
  reject unsupported plot shape or stated sloped terrain
  if invalid: preserve draft and return field/scope errors
  snapshot ⟵ reviewed answers and version metadata
  append snapshot, retaining previous confirmed briefs
  persist snapshot; report any storage failure
  return confirmed brief
```

The generation pseudocode shows where spatial construction, affordability and plot feedback are combined. Unsupported preferences remain visible for review rather than being presented as generated features.

Generate a house concept

```
GENERATE_CONCEPT(confirmedBrief)
  needs ⟵ supported room requirements from brief
  rooms ⟵ fixed-size living and room components
  place private rooms in paired hallway bays
  fill unmatched bay with laundry/storage
  connect separate kitchen to living area if applicable
  derive doors, windows and wall allowances
  compute gross width, length and circulation area
  area ⟵ rounded gross footprint area
  rate ⟵ illustrative finish rate from layout generator
  estimate ⟵ area × rate
  envelope ⟵ plot dimensions minus demo edge allowances
  orient fit comparison for the recorded road access
  flag footprint or budget conflicts without dropping rooms
  return shared geometry, style, estimate and flags
```

The plot comparison allows 1 m per edge as a demo envelope, not as a verified planning rule. East/west road access swaps frontage and depth for this comparison. Unknown terrain remains explicit; no terrain analysis is performed. The cost model uses gross area multiplied by an illustrative finish rate, with no verified quantity-based breakdown. Furniture and clearance drawings are indicative.

7.6 Worked layout and estimate

Figure 7 reproduces the spatial arrangement of the documented example. The brief describes a 20 × 30 m plot, three bedrooms, two bathrooms, traditional style and a basic finish. The generated outside footprint is 8.8 × 15.2 m. The labelled internal room dimensions illustrate the front living zone and private bays around the hallway; they should not be treated as a construction drawing.

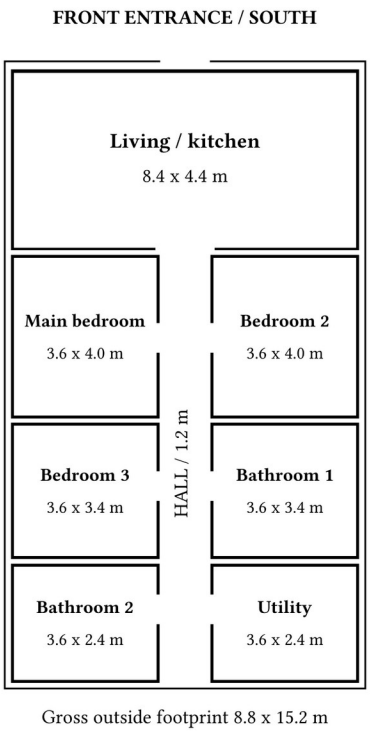


Figure 7: Schematic of the worked concept and internal room dimensions.

The gross floor area is 8.8 × 15.2 = 133.76 m². At the demo basic-finish rate of R9,500/m², the estimate is 133.76 × R9,500 = R1,270,720. Against a budget of R1,500,000, this leaves R229,280 within that illustrative calculation. The house fits the demo envelope for the example. This result depends on the stated rate and assumptions; it does not establish an approved site layout or a guaranteed construction price.

7.7 Interface design and process

The interface translates the consultation mental model into a sequence of manageable tasks. A user starts a named project, answers questions about land, budget, household, daily life, style and priorities, then reviews and confirms the brief. Conditional questions address office needs when working from home, guest needs when guests stay often, and specific access requirements. Unsupported details are recorded for review.

The confirmed project opens in Your house. Edit brief remains a separate view, addressing the recorded scrolling concern E4. Figure 8 is a schematic of the described workspace, not a screenshot. It groups exploration controls with the concept viewport and keeps estimate/conflict information and exports nearby. Review details can be collapsed; consultation actions remain available through sticky controls.

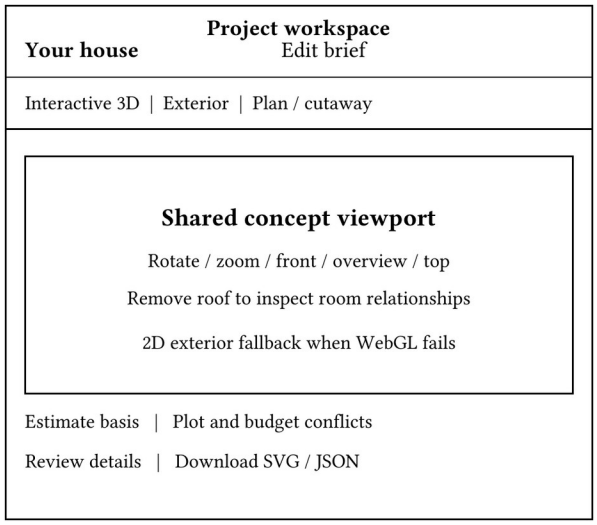


Figure 8: Interface schematic derived from the prototype workspace design.

Interactive 3D supports rotation, zoom, front/overview/top presets and roof removal. Exterior views show front, side and perspective appearances with illustrative finishes. Plan/cutaway views expose dimensions, openings and rooms. Keyboard-operable camera buttons supplement mouse and touch gestures. Loading/retry feedback, storage errors and a stale-concept message after edits communicate the system state. If WebGL fails, the user receives a 2D exterior view and a retry option.

The UI design process connects the analysis tasks to screen organisation, implements an executable view, and returns to evaluation against N01, N02 and N06. The current design decisions respond to project feedback; reduced scrolling, accessibility and user understanding still require task-based evaluation. No participant improvement score is claimed.

8 PROTOTYPE DEVELOPMENT

8.1 Prototype approach and environment

SmartHouse is an evolutionary software prototype: actual program code demonstrates key functionality and can be refined into later application increments. Its present purpose is to evaluate the consultation and concept experience before adding backend infrastructure. Table 11 describes the development and execution environment at the level supported by the design baseline [8].

Table 11: Prototype environment and tools

Area	Role in development or execution
React and Vite	Screens, shared interface components and frontend development/build workflow.
JavaScript modules	Local state adapter, consultation validation, geometry, cost feedback and export logic.

Area	Role in development or execution
Three.js and WebGL	Interactive house meshes, lighting, camera and OrbitControls.
SVG and JSON	Floor plans, exterior representations and downloadable concept information.
Browser localStorage	Device/browser-local projects, drafts, briefs and demo content.
Development tools	JavaScript build environment, ESLint, prototype test runner and Git/GitHub version control.
Execution hardware	Computer or device running a compatible browser; WebGL graphics support for 3D, with a 2D fallback.

The prototype does not require a dedicated API or database server. The evaluation environment must record the actual device, operating system and browser version before performance or compatibility claims are made. No minimum RAM, processor or GPU benchmark is established by the current reports. Internet-independent workflow tests concern a loaded application and local operations; they do not establish an offline installation or caching mechanism.

8.2 Visual implementation and lifecycle

houseModel.js constructs floor slabs, exterior walls, real window/door openings, partitions, furniture and style-dependent roof geometry. House3DView.jsx sets up the camera, lighting and OrbitControls, resizes to the viewport, exposes rotation/zoom controls and lowers walls when the roof is removed. HouseElevation provides the exterior representation. The 3D viewer is loaded as a separate module.

Models are built in the browser from the local geometry rather than downloaded from an asset server. When the view unmounts, geometries, materials, controls and renderer resources are disposed. This lifecycle supports bounded resource use while the 2D fallback keeps exploration available after a graphics failure. External photorealistic rendering, free-form room editing and a construction-ready CAD model are not implemented.

8.3 Demonstration coverage

The prototype demonstration should follow one coherent project: create a workspace, complete the consultation, review and confirm, inspect the same concept in 3D and plan views, read cost/plot feedback, revise a supported requirement and export the resulting SVG or JSON. Showing a brief revision also exposes the distinction between saving and confirming. The accompanying narrated MP4 is a separate submission with the course's three-minute maximum [9]; the report documents the design behind that workflow.

9 SOFTWARE ENGINEERING PRACTICE

9.1 Process and methods

The project follows an iterative Scrum-oriented process [1, 10]. The original proposal commits to short, time-boxed sprints with regular stakeholder feedback. Requirements are elicited from available feedback, prioritised, translated into small prototype increments and reviewed again. The design increment carries the Step 4 requirements forward through architecture, detailed objects, interface decisions and executable behaviour.

Methods include use case modelling, abstract and detailed class/object analysis, conditional questionnaires, interaction sequences, interface schematics, pseudocode and executable prototype evaluation. Figures 1 and 3 capture goals and domain relationships; Figures 4-6 refine system responsibilities and interactions. The worked example in Figure 7 makes area and cost assumptions inspectable. These methods connect design decisions to the requirements they address.

The proposal's SE methods are UML use case and class diagrams, requirements analysis, and low-fidelity prototyping [10]. Sections 3-6 apply the first two through actors, domain objects, functional and nonfunctional requirements, and written use cases. Low-fidelity wireframes remain a planned method for reviewing interface alternatives before further coding; Figure 8 is a schematic of the current workspace, not evidence that a Figma study occurred. The implemented Step 5 prototype is evolutionary code, as described in Section 8.

9.2 SE principles

User-centred design and modular architecture are the two principles named in the proposal [10]. User-centred design connects homeowner goals and the consultation mental model to labelled questions, editable summaries, preserved answers and a separate design workspace. The scrolling feedback E4 informs this separation. Independent user evaluation remains necessary to establish whether these decisions improve usability.

Modular architecture separates consultation, local persistence, layout generation, cost feedback and rendering through the responsibilities in Sections 7.1-7.3. For example, the 3D viewer consumes the shared house object instead of maintaining a separate room layout. This supports focused testing and allows later storage or rendering changes without rewriting every screen.

Abstraction separates the user-facing brief from generated geometry. Modularity and separation of concerns distinguish consultation, persistence, layout, costing and rendering. Consistency is supported by the shared house representation; explicit revision checks and failure feedback avoid false save or confirmation success. Incremental development keeps authentication and production costing outside the current implementation boundary while their requirements remain documented.

9.3 SE goals from ISO 25 010

The proposal explicitly selects usability and reliability as its ISO 25 010 quality goals [10]. This report uses the ISO/IEC 25 010:2011 terminology for those goals [11]. These are project evaluation goals, not a claim of ISO certification or demonstrated conformance.

Usability means homeowners should understand the consultation, enter their needs, interpret the proposed house and recognise the limits of its estimate. The design supports this goal through labelled units, conditional questions, review before confirmation, preserved answers, clear feedback and consistent plan/3D views. It maps to N01, N02 and N06. Evaluation should observe completion of the tasks in Section 10.2 and record errors, requests for help, task time and understanding of estimate exclusions. Keyboard and narrow-screen checks should accompany those sessions; acceptance targets must be agreed before testing.

Reliability means supported operations should behave consistently, preserve valid project state and handle failures visibly. Draft revision checks, confirmation validation, local saving, storage-error feedback and the 2D fallback support this goal, principally N03 and N04. Verification should test save/reopen, rejection of stale updates, preservation of the previous brief after failed confirmation, and continued 2D access when WebGL fails. Record failures and recovery outcomes across repeated runs. The current local prototype does not establish server availability, backup recovery or cross-device durability.

The proposal also seeks accurate recommendations. Geometry and calculation checks contribute to functional correctness, while reliability concerns consistent operation under stated conditions. Neither implies that illustrative prices are verified construction costs. The coverage in Table 12 is a starting point; measured usability and reliability results remain to be collected.

9.4 SE umbrella activities and tools

Quality assurance and testing, risk management, and Git version control are the umbrella activities committed to in the proposal [10]. They continue across requirements, design, implementation and evaluation rather than forming a single final stage.

Quality assurance and testing include design review, unit and integration checks, and planned user acceptance evaluation during the sprint cycle. The prototype suite covers local create/save/confirm/reopen tasks, incomplete/stale updates, storage failures, room relationships and selected 3D geometry invariants [7, 8]. Build and ESLint checks support code quality. User acceptance and professional review are still required; automated coverage does not establish stakeholder acceptance.

Risk management should keep a register of the risks in Section 9.5, their likelihood and impact, mitigation actions, responsible team member and review status. The team should revisit it during sprint planning and review, especially when adding layout patterns, costing data or backend integration. Git/GitHub support configuration and version control by retaining code and report history; the planned branching and review workflow should associate each change with its requirement and verification evidence.

Traceability links F01-F04 and N04 to the consultation/local store; F05-F06 to layout and conflict logic; F07-F08 and N03 to shared geometry and views; F09 to estimate feedback; F10 to brief snapshots; and F11 to SVG/JSON exports. F12 remains local demo maintenance. F13 stays in the later platform scope. Partial statuses in Table 7 prevent documentation from overstating the implementation.

The proposal lists Jira or Trello for backlog and sprint tracking, Figma for wireframes, Visual Studio Code for development, and MySQL or PostgreSQL for persistent platform data [10]. These are planned tool choices, not confirmation that every tool has been used. Table 11 records the current prototype environment; its active persistence remains browser localStorage rather than a database. Documentation and requirements traceability provide additional umbrella support through this report and its linked models.

9.5 Risks and scope control

The principal risks are an attractive model being mistaken for a build-ready design, inaccurate pricing assumptions, a narrow layout pattern failing diverse needs, and complexity expanding beyond the academic timeline. Mitigations are explicit concept status, transparent demo rates, recorded unsupported requirements, representative task evaluation and incremental scope decisions. Local storage also limits durability and cross-device access. Accessibility and professional compliance must be assessed rather than inferred from an illustrative model.

10 VERIFICATION AND EVOLUTION

10.1 Engineering verification

Table 12 records verification coverage described for the current prototype [7, 8]. The design baseline identifies eight test cases in the prototype runner, together with frontend build and ESLint checks. This is coverage evidence, not a new test execution log or a claim of measured browser performance. Browser interaction and professional design review remain separate evaluation tasks.

Table 12: Recorded prototype verification coverage

Area	Checks described in the design baseline
Local workflow	Create, save, confirm and reopen without API calls; stale revisions, incomplete briefs and storage failure feedback.
Layout geometry	Supported bedroom/bathroom combinations; finite dimensions; footprint bounds; no room overlap; hallway access; exterior windows.
3D consistency	Room-floor count matches plan; real mesh openings; roof height/style checks; interior survives roof removal.

Area	Checks described in the design baseline
Build and code quality	Frontend build, ESLint and eight prototype test cases; browser/professional evaluation remains separate.

10.2 Research and acceptance plan

A proposed next study should recruit prospective homeowners with varied planning experience, including household, home-working and access needs where participants choose to discuss them. Short semi-structured interviews and task observation should use consent and anonymised IDs. Participants would create a project, complete a brief, interpret a room arrangement, explain an estimate, revise a need and export a concept. Record completion, errors, questions and observed effort separately from researcher interpretation. A designer and cost practitioner should review the spatial and pricing assumptions. These activities are proposed, not completed findings.

For each future research record, capture the date and method, anonymised participant ID and relevant planning experience, task/device/context, exact response or observed behaviour, interpretation and alternative explanation, related requirement, and consent/record reference. Avoid unnecessary personal identifiers or sensitive household information.

10.3 Limitations and future increments

The active generator uses one adaptable layout pattern, fixed component sizes and illustrative area rates. It does not search arbitrary layouts, interpret the imported floor-plan image dataset, enforce all stated lifestyle priorities, verify accessibility clearances, or perform structural/regulatory checks. Local storage is device-specific and has no authentication or cross-device synchronisation. Brief snapshots persist, but exact geometry and pricing replay is not guaranteed.

Next prototype work includes additional layout patterns, more flexible room sizing, clearer material options, stronger circulation checks and user feedback. Later platform work includes persisted geometry/pricing snapshots, validated configuration releases, quantity-based costing, protected projects, professional review and downloadable concept packages. Backend implementation is deferred until the prototype workflow is agreed. The future REQUIREMENTS, ARCHITECTURE and ERD specifications therefore describe later work beyond the active frontend baseline [8].

11 CONCLUSIONS

SmartHouse provides a guided architectural planning workspace in which users describe needs, confirm a brief, explore a coherent house concept, examine a demo estimate and revise. The user and mental models establish traceable data, functional, behavioural and quality requirements. The detailed design maps those requirements to local project records, shared geometry, interface components and explicit save, confirmation and generation rules.

The executable frontend prototype demonstrates this workflow through browser-local persistence, rule-based layouts, SVG plans, exterior views, interactive 3D and concept exports. Its current limits remain explicit: personalisation uses a constrained pattern, costs are illustrative, and professional approval and protected accounts are future work. The next priority is independent user and domain-expert evaluation of the consultation questions, spatial comprehension, estimate expectations and navigation, followed by targeted design increments.

REFERENCES

[1] FivePlusOne Devs, “SmartHouse Revised Project Proposal,” CSC 312 project document, 2026. Repository: docs/SmartHouse_Revised_Project_Proposal.md.

[2] CSC 312 course material, “Step 4 User Model Requirements and Analysis,” assignment brief, 2026. Includes Chapter 15 User Interface Design guidance.

[3] SmartHouse project discussion, “Product requirements and prototype feedback,” 10-11 October 2026. Recorded project-owner/team-representative messages; not an independent participant study.

[4] Planner 5D, “House Design Software and Home Design in 3D,” 2026. [Online]. Available: https://planner5d.com/. Desk-review access: 11 October 2026.

[5] Floorplanner, “Online Floor Planner and 3D Room Planner Tool,” 2026. [Online]. Available: https://floorplanner.com/. Desk-review access: 11 October 2026.

[6] CSC 312 course material, “ACM SigConf format for SE Reports 2026,” supplied Word report-format template, 2026.

[7] FivePlusOne Devs, “SmartHouse frontend prototype and test suite,” 2026. Repository paths: client/src and client/tests/prototype.test.cjs. Baseline dated 11 October 2026.

[8] FivePlusOne Devs, “SmartHouse Design Model,” 11 October 2026. Design baseline citing docs/PROTOTYPE_PROGRESS.md; client/src/demo/store.js, layout.js and houseModel.js; ProjectsPage.jsx, PrototypeConcept.jsx, House3DView.jsx and HouseElevation.jsx; and future docs/REQUIREMENTS.md, ARCHITECTURE.md and ERD.md.

[9] CSC 312 course material, “Step 5 SE Group Project Design Model plus a Prototype,” assignment instructions, 2026. Design Concepts, Chapter 12; User Interface Design and Process, Chapter 15; PDF report and narrated MP4 demonstration.

[10] FivePlusOne Devs, “Software Engineering Project Proposal: Smart House Design & Cost Recommendation System,” CSC 312 project proposal, Section 3, pp. 1-2.

[11] ISO/IEC, “ISO/IEC 25 010:2011: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models,” 2011. [Online]. Available: https://www.iso.org/standard/35 733.html. Terminology basis for the proposal's usability and reliability goals; this edition has been superseded.